DATABASE WALKTHROUGH

Picturehouse Jaipur

MySQL schema, inserts, joins, transactions, and expected query results

A practical handout for demonstrating the project database in MySQL Workbench. It explains what is stored, where the tables connect, how the application writes data, and how to run safe classroom examples.

WHAT YOU CAN SHOW

Topic	What the teacher can inspect
Full live schema	All 15 project tables, their actual CREATE TABLE statements, keys, constraints, and indexes.
Starter data	Development catalog rows and how to inspect the movie, theatre, showtime, and seat data.
SQL practice	Names beginning with D, a multi-table JOIN, seat counts, and a booking summary.
Transactions	The app's payment and ride transaction flow, plus an isolated INSERT/UPDATE/ROLLBACK lab.
Expected results	Exact expected rows for the small demo fixture; notes where live results depend on your database.

Important: the app creates or updates tables from Java/JPA entity classes. The exact current CREATE TABLE text and all current production-like rows live in your local MySQL server, so use the export steps in this handout to produce a true full dump. The sample transaction lab is intentionally rolled back and leaves no demo data behind.

Prepared from the project source and the development catalog seeder. The local server's private account data and credentials are not included.

1. Open the right database and run queries

In MySQL Workbench, open your saved Local MySQL connection. The SQL editor tab is the white editor area; type or paste SQL there. Select only the statement you want to run, then click the lightning-bolt Run button (or press Ctrl+Enter). The lower Output/Result tabs show success messages and returned rows.

The project schema is named **picturehouse**. Always begin a new query tab with this, or qualify every table name with the schema. That avoids the earlier 'No database selected' message.

```
USE picturehouse;
SELECT DATABASE() AS current_schema, VERSION() AS mysql_version;
SHOW TABLES;
```

Expected first query: current_schema is **picturehouse**; mysql_version should be your installed MySQL 8.4.x. SHOW TABLES should list the 15 app tables listed in Section 2.

For a read-only first look at data:

```
SELECT id, title, language, genre, certification, duration_minutes, active
FROM picturehouse.movies
ORDER BY id;
```

A development database seeded by this project has four sample movies: The Last Maharaja, Dhoom Dhaam Jaipur, The Silent Fort, and Chai & Chandni. They are fictional development-catalog entries. Your local results may differ if you have edited the catalog or imported other data.

This seeder only runs outside the prod profile and only when both movies and theatres are empty. It makes future showtimes for today through the next seven days at runtime, so those dates and rows change with the calendar. It is not a fixed SQL dump.

2. The 15 tables and how they fit together

The database separates core things into tables: a movie is stored once; a theatre has screens; a screen has showtimes; seats belong to a showtime. Booking and ride workflows use their own tables. The following list reflects the project entities and Workbench table list.

Area	Tables	Purpose
Accounts	accounts	Application user identity, email/mobile, role, and verification state.
Movie catalog	movies, theatres, theatre_movies	Movie and theatre details; theatre_movies links which theatres offer which movies.
Schedule and seats	screens, showtimes, showtime_seats	A theatre's auditorium, scheduled movie sessions, and per-show seat inventory.
Ticket checkout	razorpay_orders, ticket_bookings	Payment-provider order state and the issued ticket/receipt snapshot.
Earlier booking model	bookings	Legacy/domain booking entity retained in the schema; current checkout issues ticket_bookings.
Ride share	ride_posts, ride_join_requests, ride_chat_messages	Ride listings, passenger requests, and their conversation messages.
Earlier ride model	carpool_rides	Legacy ride entity retained in schema; current ride portal uses ride_posts.
Theatre access	theatre_owner_memberships	Clerk user-to-theatre access request/approval state.

Relationships to point out: theatres 1-to-many screens; movies 1-to-many showtimes; screens 1-to-many showtimes; showtimes 1-to-many showtime_seats. A theatre and movie can be many-to-many through theatre_movies. ride_posts 1-to-many ride_join_requests; each ride_join_request 1-to-many ride_chat_messages.

3. See the complete live database definition

The authoritative table code is MySQL's current DDL. Run each SHOW CREATE TABLE command. Workbench returns a result row whose Create Table column contains the full definition, including column types, PRIMARY KEY, FOREIGN KEY, UNIQUE, indexes, and engine settings. These statements explicitly name the schema, so no default schema is needed.

```
SHOW CREATE TABLE picturehouse.accounts;
SHOW CREATE TABLE picturehouse.bookings;
SHOW CREATE TABLE picturehouse.carpool_rides;
SHOW CREATE TABLE picturehouse.movies;
SHOW CREATE TABLE picturehouse.razorpay_orders;
SHOW CREATE TABLE picturehouse.ride_chat_messages;
SHOW CREATE TABLE picturehouse.ride_join_requests;
SHOW CREATE TABLE picturehouse.ride_posts;
SHOW CREATE TABLE picturehouse.screens;
SHOW CREATE TABLE picturehouse.showtime_seats;
SHOW CREATE TABLE picturehouse.showtimes;
SHOW CREATE TABLE picturehouse.theatre_movies;
SHOW CREATE TABLE picturehouse.theatre_owner_memberships;
SHOW CREATE TABLE picturehouse.theatres;
SHOW CREATE TABLE picturehouse.ticket_bookings;
```

To save the entire schema *and current rows* as one SQL file in Workbench:

Step	Action
1	Go to Server > Data Export.
2	Select the picturehouse schema and all its tables.
3	Choose 'Dump Structure and Data' (not structure-only). Choose 'Export to Self-Contained File'.
4	Choose a file location you can find, such as Documents\picturehouse_full_backup.sql.
5	Click Start Export. Keep the file private because it may contain customer data, emails, and payment references.
6	For a classroom copy, export a sanitized copy or use the rollback-only demo in Section 7. Never share passwords or real customer details.

A SQL editor tab containing SELECT queries is not a database dump. The Data Export tool creates the portable file containing DDL and INSERT statements for the rows that exist at export time. If teacher needs to run the project, they also need a compatible MySQL server; do not import over a database that contains important data.

List database-level constraints and foreign keys

```
SELECT TABLE_NAME, CONSTRAINT_NAME, CONSTRAINT_TYPE
FROM information_schema.TABLE_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = 'picturehouse'
ORDER BY TABLE_NAME, CONSTRAINT_TYPE, CONSTRAINT_NAME;

SELECT TABLE_NAME, COLUMN_NAME, CONSTRAINT_NAME,
       REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
FROM information_schema.KEY_COLUMN_USAGE
WHERE CONSTRAINT_SCHEMA = 'picturehouse'
  AND REFERENCED_TABLE_NAME IS NOT NULL
ORDER BY TABLE_NAME, CONSTRAINT_NAME, ORDINAL_POSITION;
```

4. Keys, constraints, and normalization

The entities use generated numeric primary keys. Foreign keys connect rows that must refer to a parent row. Unique constraints stop duplicate business identifiers. The live MySQL server is the final authority: use SHOW CREATE TABLE and information_schema above to verify the exact key names and whether an annotation became a SQL-level constraint.

Rule	Examples in this project	Why it matters
Primary key	id on most entities; theatre_id + movie_id on theatre_movies where mapped as a join entity.	Identifies one row and is the target for references.
Foreign key	screens.theatre_id; showtimes.movie_id and screen_id; showtime_seats.showtime_id; ride_join_requests.ride_post_id; ride_chat_messages.join_request_id.	Prevents references to a missing parent.
Unique	accounts email/mobile; one screen name per theatre; one seat code per showtime; one join request per ride and requester; unique ticket/order/reference codes.	Prevents duplicate identities, inventory seats, requests, or receipts.
Not null / required	Explicit @Column(nullable=false), relation optional=false, and required IDs/codes on entities.	Rejects incomplete rows at the database layer where mapped.
Application validation	@NotBlank, @NotNull, @Min/@Max, @Pattern on request DTOs and entities.	Rejects bad user input in Spring; some validation annotations are not SQL CHECK constraints.
Concurrency/version	@Version on showtimes and ride_posts; pessimistic FOR UPDATE in ride acceptance and seat locking in booking.	Stops overlapping updates from silently overwriting each other.

Normalization: separating movies, theatres, screens, showtimes, and seats avoids repeating the full movie and theatre data on every seat. Theatre/movie association uses a link table. This is a largely normalized relational core and supports a 3NF-style explanation: non-key attributes belong with the key of their own entity. Ticket/checkout tables also keep a few descriptive snapshots on purpose, so describe the overall schema as normalized with selected historical denormalization, rather than claiming every column everywhere is strictly 3NF.

Check constraints carefully: Java Bean Validation does not automatically prove that MySQL has a CHECK rule. To show your teacher what the database itself enforces, use SHOW CREATE TABLE and the information_schema queries. Database UNIQUE/FK/NOT NULL constraints are different from validation in Java.

5. Query 1 - list names beginning with D

For movie titles, LIKE 'D%' means the first character is D. MySQL matching is often case-insensitive under the table collation; LOWER makes the intent explicit. This query reads live catalog data and does not change anything.

```
SELECT title AS name
FROM picturehouse.movies
WHERE LOWER(title) LIKE 'd%'
ORDER BY title;
```

On the development catalog shown in the project screenshots, this includes **Dhoom Dhaam Jaipur**. Other results depend on the titles currently stored. If teacher means people/accounts rather than movie names, use:

```
SELECT display_name AS name
FROM picturehouse.accounts
WHERE LOWER(display_name) LIKE 'd%'
ORDER BY display_name;
```

That second query returns actual account names from your database. Use a sanitized copy for a classroom demonstration; do not display or export real users' private data unnecessarily.

6. Query 2 - JOIN movies, theatres, screens, showtimes, and seats

A JOIN combines rows by matching their related key columns. Here the path follows the ER relationships: showtime -> movie, showtime -> screen -> theatre, and seat -> showtime. The aggregates count total, booked, and free seats for each session.

```
SELECT m.title AS movie,
       t.name AS theatre,
       DATE_FORMAT(st.starts_at, '%Y-%m-%d %H:%i') AS starts_at,
       sc.name AS screen,
       COUNT(ss.id) AS seats_total,
       SUM(CASE WHEN ss.booked = 1 THEN 1 ELSE 0 END) AS seats_booked,
       SUM(CASE WHEN ss.booked = 0 THEN 1 ELSE 0 END) AS seats_available
FROM picturehouse.showtimes st
JOIN picturehouse.movies m ON m.id = st.movie_id
JOIN picturehouse.screens sc ON sc.id = st.screen_id
JOIN picturehouse.theatres t ON t.id = sc.theatre_id
LEFT JOIN picturehouse.showtime_seats ss ON ss.showtime_id = st.id
WHERE st.starts_at >= NOW()
GROUP BY st.id, m.title, t.name, st.starts_at, sc.name
ORDER BY st.starts_at, t.name
LIMIT 20;
```

Expected columns: movie, theatre, starts_at, screen, seats_total, seats_booked, seats_available. The rows and times change as data and time change. LEFT JOIN keeps a showtime visible even if its seat inventory is incomplete; COUNT will then be zero.

A second JOIN: ticket totals and rupee revenue

```
SELECT theatre_name,
       movie_title,
       COUNT(*) AS tickets,
       ROUND(SUM(amount_paise) / 100.0, 2) AS revenue_rupees
FROM picturehouse.ticket_bookings
GROUP BY theatre_name, movie_title
ORDER BY tickets DESC, theatre_name, movie_title;
```

The application stores money in paise as an integer to avoid floating-point rounding. Dividing by 100 converts to rupees for display. If the table is empty, this query returns an empty result; otherwise it summarizes your actual saved bookings.

7. Safe INSERT, UPDATE, JOIN, and ROLLBACK classroom lab

This example writes one clearly named fictional movie, theatre, screen, showtime, and three seat rows inside a transaction. Run Part A, then the queries in Part B, then Part C in the **same Workbench connection**. Part C rolls everything back. Do not click Commit. The date 2099 keeps this fixture separate from current screenings.

Part A - begin transaction and insert temporary rows

```
USE picturehouse;
START TRANSACTION;

INSERT INTO picturehouse.movies
  (title, language, genre, certification, duration_minutes, synopsis, active)
VALUES
  ('D Demo Movie', 'Hindi', 'Classroom Demo', 'U', 90,
   'Temporary classroom row; it will be rolled back.', 1);
SET @demo_movie_id = LAST_INSERT_ID();

INSERT INTO picturehouse.theatres
  (name, area, address, city, theatre_type, parking_info, active)
VALUES
  ('Demo Central 2099', 'Demo Area', '1 Demo Road', 'Jaipur',
   'Classroom', 'Not applicable', 1);
SET @demo_theatre_id = LAST_INSERT_ID();

INSERT INTO picturehouse.screens (theatre_id, name, seat_count)
VALUES (@demo_theatre_id, 'Teacher Demo Screen 2099', 3);
SET @demo_screen_id = LAST_INSERT_ID();

INSERT INTO picturehouse.showtimes
  (movie_id, screen_id, starts_at, format, on_sale, `version`)
VALUES
  (@demo_movie_id, @demo_screen_id, '2099-10-25 18:00:00', '2D Hindi', 1, 0);
SET @demo_showtime_id = LAST_INSERT_ID();

INSERT INTO picturehouse.showtime_seats (showtime_id, seat_code, booked)
VALUES (@demo_showtime_id, 'A1', 0),
       (@demo_showtime_id, 'A2', 1),
       (@demo_showtime_id, 'A3', 0);
```

Workbench may display the status for each INSERT in Output. The @demo_* variables belong to this connection; keep the same query tab/connection open until Part C finishes.

Part B - run the queries and compare results

```
-- Names beginning with D (the exact fixture result is D Demo Movie)
SELECT title AS name
FROM picturehouse.movies
WHERE title LIKE 'D%'
ORDER BY title;

-- A multi-table JOIN for only the demo screening
SELECT m.title AS movie, t.name AS theatre, sc.name AS screen,
       DATE_FORMAT(st.starts_at, '%Y-%m-%d %H:%i') AS starts_at
FROM picturehouse.showtimes st
JOIN picturehouse.movies m ON m.id = st.movie_id
JOIN picturehouse.screens sc ON sc.id = st.screen_id
JOIN picturehouse.theatres t ON t.id = sc.theatre_id
WHERE st.id = @demo_showtime_id;

-- Count total, booked, and available seats
SELECT COUNT(*) AS total_seats,
       SUM(booked = 1) AS booked_seats,
       SUM(booked = 0) AS available_seats
FROM picturehouse.showtime_seats
WHERE showtime_id = @demo_showtime_id;

-- UPDATE inside the open transaction
UPDATE picturehouse.movies
SET synopsis = 'UPDATED inside the demo transaction'
WHERE id = @demo_movie_id;
SELECT title, synopsis
FROM picturehouse.movies
WHERE id = @demo_movie_id;
```

Query	Expected demo result
Name begins D	The result contains D Demo Movie. Dhoom Dhaam Jaipur may also appear if the development row is present; other existing D titles can appear too.
Four-table showtime JOIN	One row: D Demo Movie | Demo Central 2099 | Teacher Demo Screen 2099 | 2099-10-25 18:00
Seat aggregate	total_seats = 3; booked_seats = 1; available_seats = 2
After UPDATE	One row with title D Demo Movie and synopsis UPDATED inside the demo transaction.

Part C - undo the lab and prove the temporary rows are gone

```
ROLLBACK;

SELECT COUNT(*) AS demo_movie_rows_remaining
FROM picturehouse.movies
WHERE title = 'D Demo Movie';
```

Expected final value: demo_movie_rows_remaining = 0. ROLLBACK undoes the five INSERTs and the UPDATE together. MySQL may still leave gaps in auto-increment IDs after a rollback; that is normal and does not mean the rows remained.

If you accidentally disconnect before ROLLBACK, MySQL normally rolls back an uncommitted transaction when its connection closes. Still, run Part C in the same connection so you can explicitly demonstrate the result.

8. Where the application uses transactions

A transaction groups database changes into one unit. If a required step fails, the database can roll back the changes so it does not keep only half of a booking or ride update. Spring marks these methods with @Transactional.

Workflow in code	What is grouped together	Database behavior
Payment order creation - RazorpayController.createOrder	Validates selected seats/showtime, requests provider order, stores a RazorpayOrder.	A DB rollback can undo the local order row. The external Razorpay request is not part of a MySQL transaction.
Payment verification - RazorpayController.verifyPayment	Checks payment signature/state, locks and marks seats, updates the order, inserts ticket_bookings.	Seat/order/ticket writes commit together. If seats are no longer available, the app attempts a refund/compensation.

Workflow in code	What is grouped together	Database behavior
Ride join - RideController.requestToJoin	Checks the ride and creates/finds the passenger request.	Unique ride/requester key helps stop duplicate requests.
Accept ride request - RideController.accept	Locks request and ride rows, checks status/capacity, reserves a seat, accepts request.	Either the seat decrement and acceptance both commit, or neither does.
Messages and read views	Send/decline are transactional; conversation and message list reads use readOnly transactions.	Writes are grouped; read-only transactions communicate intent and can reduce accidental writes.

Important distinction for the teacher: MySQL can atomically commit database changes, but it cannot atomically include Gmail, Razorpay, or another external service in that same local transaction. The app uses checks and compensating actions (such as refunding a captured payment if seat allocation fails). A production system commonly adds an outbox/retry workflow for stronger reliability.

How the transaction is enforced in SQL

In the classroom lab, START TRANSACTION opens a unit of work; INSERT and UPDATE are pending changes; ROLLBACK discards them. COMMIT would make them permanent. In the application, Spring opens/commits/rolls back around @Transactional service/controller work, so the Java code normally does not issue START TRANSACTION manually.

```
START TRANSACTION;
UPDATE picturehouse.showtime_seats
SET booked = 1
WHERE showtime_id = 123 AND seat_code = 'A1';
-- Inspect or perform the remaining related writes here.
ROLLBACK;  -- practice only: undo changes
-- Use COMMIT only when you intentionally want permanent changes.
```

Do not run the final illustrative UPDATE against a real seat outside a transaction. Use the Section 7 fixture for the safe demonstration.

9. How to explain a query result

SQL piece	Plain-language explanation
SELECT	Which columns/results to display.
FROM	The starting table.
JOIN ... ON	How a row in one table matches its related row in another table.
WHERE	Which rows should be included before grouping.
GROUP BY	Combine rows that share the listed values so COUNT/SUM can summarize them.
COUNT / SUM	Count matching rows or add values; here they count all/booked/available seats.
ORDER BY	Sort the final result.
LIMIT	Cap the number of returned rows.
START TRANSACTION / COMMIT / ROLLBACK	Start a group of changes, make them permanent, or undo them.

Useful presentation sequence

1. Show picturehouse in the Workbench SCHEMAS panel and run SELECT DATABASE(), VERSION().

2. Run SHOW TABLES, then SHOW CREATE TABLE for movies, showtimes, showtime_seats, ride_posts, and ticket_bookings.

3. Run the D-title query, then explain the multi-table JOIN by following each foreign key.

4. Run the demo INSERTs, the demo JOIN/count/UPDATE queries, and the final ROLLBACK. Point out the remaining count is zero.

5. Explain @Transactional in the payment and ride flows, and show how row locks/unique keys protect seats and requests.

6. If asked for every current row and every CREATE statement, use Workbench Server > Data Export and choose Dump Structure and Data. Use a sanitized export for sharing.

10. Project implementation notes and source locations

The schema comes from Spring Boot + Spring Data JPA/Hibernate entity mappings, with the MySQL Connector/J driver. Hibernate's configured schema update mode can create/update the local schema from entities. Runtime INSERT/UPDATE/DELETE statements are generally generated by Hibernate when repositories save entities; they are not all collected into one hand-written .sql file.

Feature	Project source to show
Movie/theatre/showtime/seat mappings	backend/src/main/java/in/picturehouse/jaipur/domain/Movie.java, Theatre.java, Screen.java, Showtime.java, ShowtimeSeat.java
Development seed catalog	backend/src/main/java/in/picturehouse/jaipur/config/DevelopmentCatalogSeeder.java
Ticket payment transaction	backend/src/main/java/in/picturehouse/jaipur/api/RazorpayController.java
Ride request and acceptance transactions	backend/src/main/java/in/picturehouse/jaipur/api/RideController.java
Workbench SQL inspection starter	docs/mysql_teacher_demo.sql

Current data changes: the catalog seeder exits if movies or theatres already contain rows. Therefore a fresh seed is not automatically added to a database that already has catalog data. Showtimes in the development seed are generated for today plus seven days and skip times already passed; their rows are date-dependent.

Privacy and accuracy: the live database may contain real emails, booking identifiers, ticket codes, and payment references. Export only when needed, keep the dump private, and make a sanitized classroom copy. The sample output values in Section 7 are deterministic because they come from the temporary fixture; output from live queries can differ as rows change.